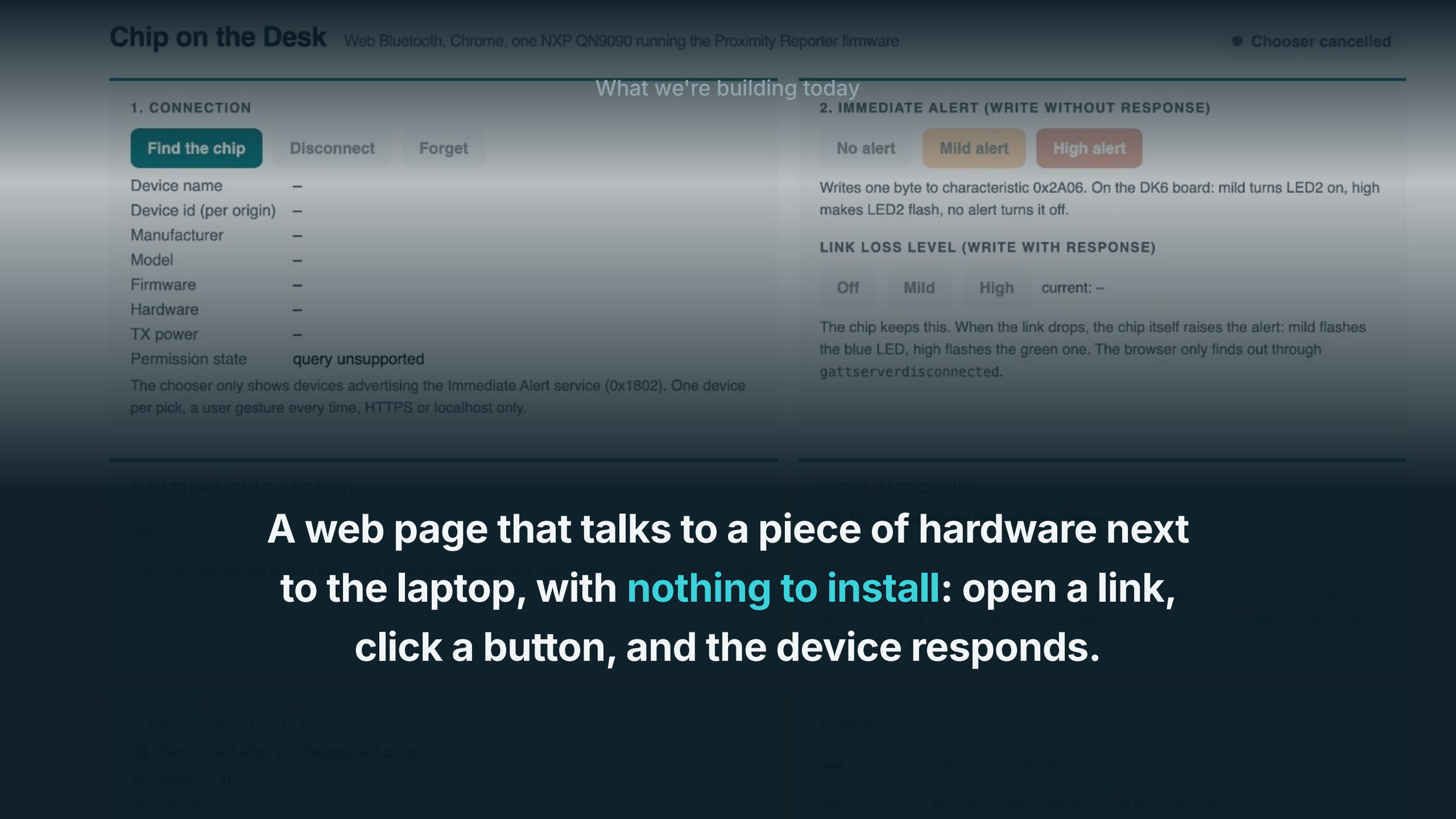
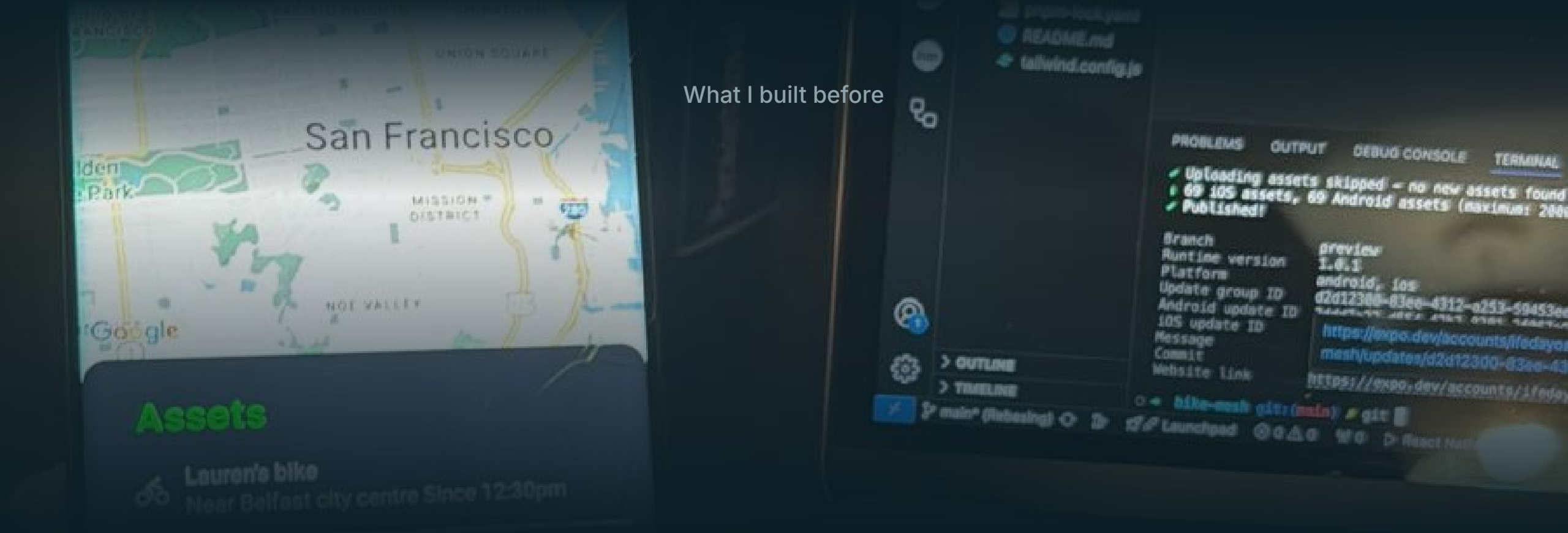

Making Web Bluetooth Work
A field report from a chip on the desk
Ifedayo Agboola · XtremeJS 2026



A web page that talks to a piece of hardware next to the laptop, with nothing to install: open a link, click a button, and the device responds.

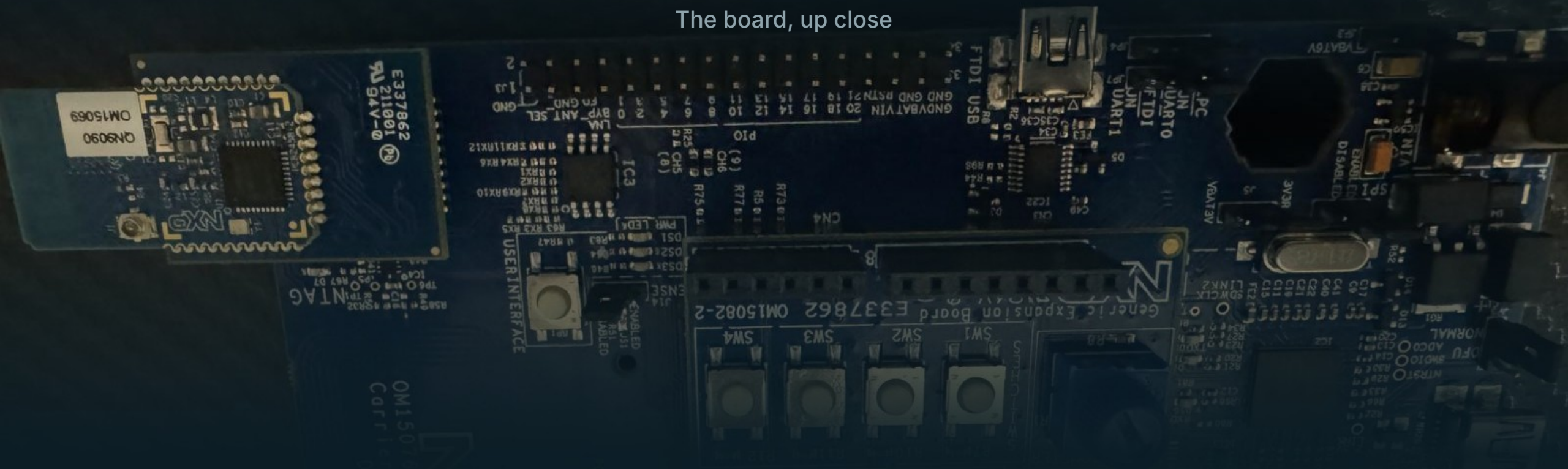
What I built before



A bike tracker: a tag that shouted its name into the air ten times a second, and a phone app that heard twenty bikes at once without connecting to any of them.

Last time a phone app listened to a chip that shouted; this time a browser tab talks to a chip that holds a conversation, and both halves had to be rewritten.

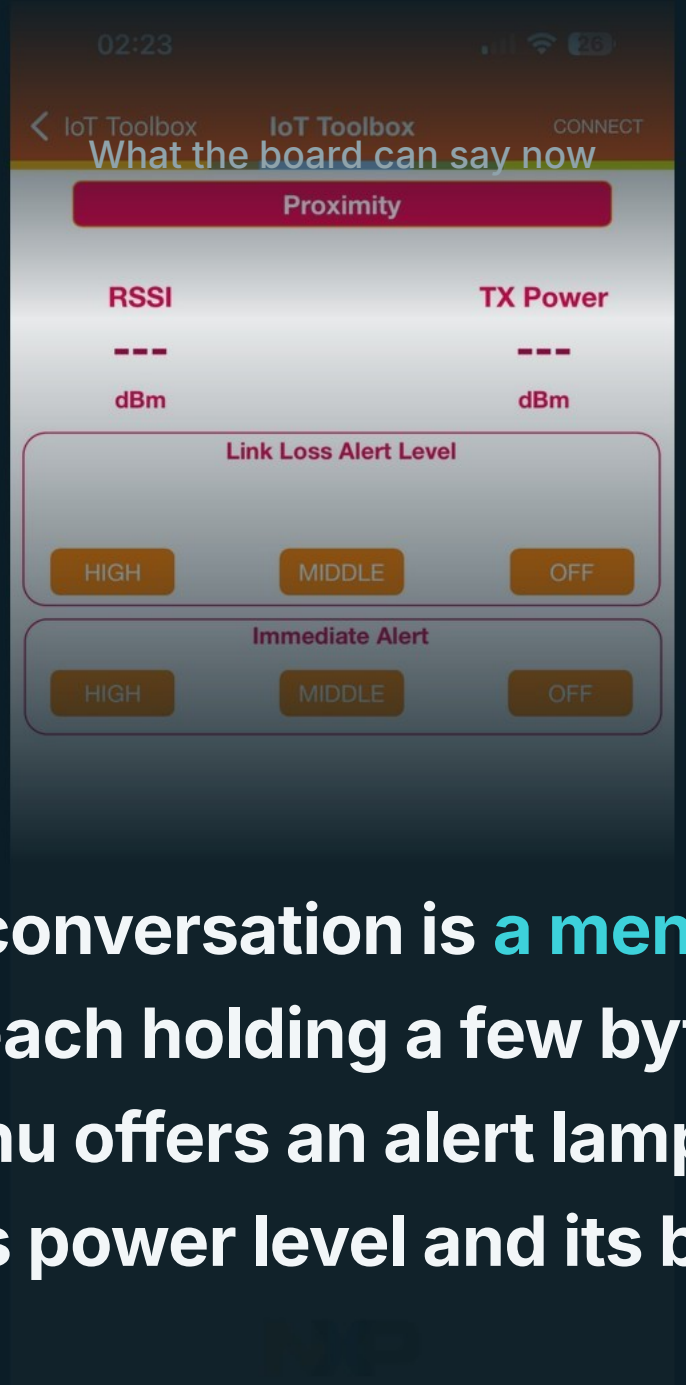
The NXP QN9090 is a Bluetooth 5 radio and a small computer in one fingernail-sized chip, on a kit with buttons, four lamps and a USB port that powers it and loads programs onto it.



The board, up close

The blue module in the corner is the chip, and the row of buttons and lamps beside it is what we will be watching.

The program on a chip is called firmware, and because a browser can only hold a conversation, the chip was given the conversation program over USB with one click.



A Bluetooth conversation is a menu of sections and items, each holding a few bytes, and this board's menu offers an alert lamp, a link-loss rule, its power level and its battery.

Web Bluetooth is the browser's built-in way to talk to nearby Bluetooth devices in plain JavaScript, turning "download our app" into "open this link".

Who already uses it

LEGO Education's classroom app, Home Assistant's Wi-Fi setup, the Bangle.js watch app store, Ledger's Nano X wallet and a Niimbot label-printing app all ship on it today: set up, control, install, sign and print, from a link.

The user clicks, the browser shows a list and the user picks exactly one device over HTTPS, so every Web Bluetooth product starts with a button that says "Connect your device".

Seven lines of JavaScript

```
const device = await navigator.bluetooth.requestDevice({
  filters: [{ services: ['immediate_alert'] }],
  optionalServices: ['link_loss', 'tx_power', 'battery_service']
});
const server  = await device.gatt.connect();
const service = await server.getPrimaryService('immediate_alert');
const alert   = await service.getCharacteristic('alert_level');
await alert.writeValueWithoutResponse(Uint8Array.of(2));
```

Ask for a device, connect, open the menu section, pick the item and send the bytes, declaring every section you will use up front.

Demo

Click, choose, connect, send one byte, and watch the board.

Demo one: tab to lamp

```
NetworkError: GATT operation already in progress
```

The radio answers one question at a time, and Chrome refuses a second question instead of queueing it, an open issue in the standard since 2015.

The queue

```
class GattQueue {
  tail = Promise.resolve();
  run(op) {
    const next = this.tail.then(op, op);
    this.tail = next.catch(() => {});   // a failure never breaks the chain
    return next;
  }
}
const gatt = new GattQueue();
await gatt.run(() => battery.readValue());
```

Every read, write and connect joins one line and waits its turn, and a failure never breaks the chain.

Links drop late and silently, so after the one event the browser fires you open the menu again, time out every connect, and retry at 1, 2, 4, 8 and 16 seconds before stopping.

Demo

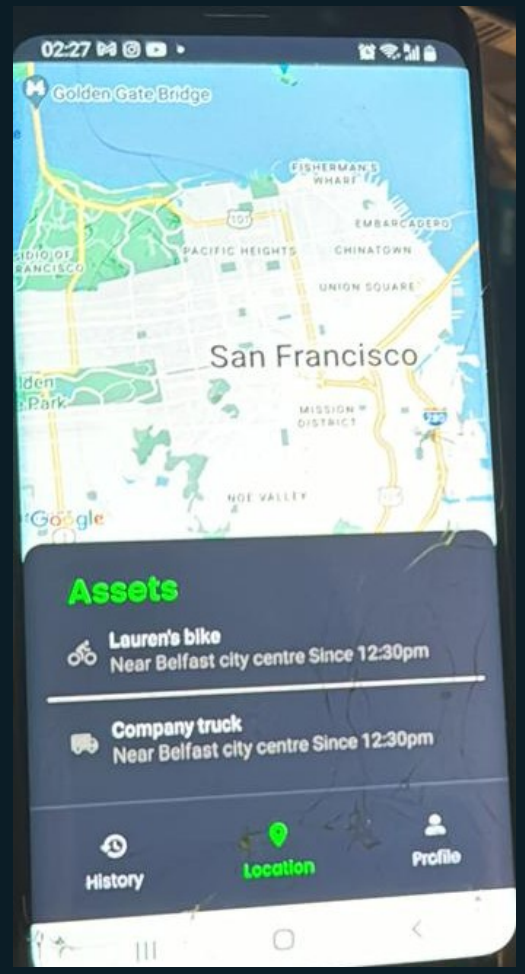
Eight questions with the queue off, eight with it on, then switch the laptop's Bluetooth off.

Demo two: kill the line

The number the browser won't give you

```
-65 -72 -78 -70 -85 -68 -92 -74 -69 -88
```

The tag sat still while the phone read these ten values, yet after you connect the browser gives you no signal strength at all, because listening without connecting is still behind a Chrome flag.



Where it works, November 2026

Chrome | Edge | Samsung Internet | Firefox | Safari

It works in Chrome and Edge on Windows, Mac, Chromebook and Android, with Linux behind a flag, while Firefox and Safari have no plan to ship it, so there is nothing on iPhone or iPad.

Build it when the device is next to the person and the job is to set it up, read it or control it, and build it with a feature check, one queue, a timeout, a reconnect plan, a log and no secrets in the page.

Two browser engines have said no for years, but the scanning spec got a fresh draft in June 2026, and a chip that needed a compiled app in 2023 needed only a text file this morning.



A bare board learned to hold a conversation, seven lines connected a browser to it, a queue, a timeout and a reconnect plan made it survive, and a lamp on the desk moved from a tab.

Thank you.

Demo page and slides:

github.com/ifedayoagboola/chip-on-the-desk

The tracker write-up:

dev.to/blackscripts

linkedin.com/in/ifedayoagboola



demo page and slides

Ifedayo Agboola · XtremeJS 2026